

# What `sync()` Certifies: A Consumer-Relative, Witnessed Reading of ZooKeeper Staleness

Andrew H. Bond

Department of Computer Engineering  
San José State University, San José, CA, USA  
andrew.bond@sjsu.edu

**Abstract**—A ZooKeeper client may read from any server, and a follower answers from its own applied state—so a local read can be stale. ZooKeeper offers `sync()` to force a read to reflect the latest committed state. We use this familiar mechanism as a compact introduction to *Observation Theory* (OT): the thesis that reliability is a property of the relation between a system’s error and *what a particular consumer reads*, and that any honest freshness claim needs an independent *witness*. In OT terms a local read is a *certificate* (this follower is fresh enough), the ZooKeeper transaction id (`zxid`) is its *witness*, and `sync()` is a witnessed correction. We measure, on a three-node ensemble under a pre-registration whose pass/fail bars are fixed before the runs, the *false-clear rate* of the local-read certificate. The result is a clean statement of consumer-relativity: on the *same* lagging follower, a hot-footprint reader false-clears  $\approx 99\%$  of the time while a cold-footprint reader reads  $\approx 1\%$  stale; graded against the `zxid`, `sync()` drives the false-clear to zero. Staleness is thus a property of the reader’s footprint, not of the replica—and `sync()` certifies not “the follower is caught up” but “the consumer’s footprint is caught up.” We then propose and measure a client-side fix, `read_fresh`: a reader declares the `zxid` watermark it needs, the read is certified locally when the follower already satisfies it, and `sync()` is paid only otherwise. It is a *sound witness*—false-clear zero at every tolerance—and its cost traces the follower-lag CDF between the two blind endpoints (naive and `sync()`-always); a self-verifying use-case notebook reproduces the result on a live ensemble.

**Index Terms**—ZooKeeper, staleness, read consistency, `sync`, observation theory, pre-registration.

## I. INTRODUCTION

ZooKeeper [1] serves reads locally: a client connected to a follower is answered from that follower’s applied state. Writes are linearizable through the leader via Zab [2], but a local read is not—it reflects whatever the follower has applied, which may trail the leader. For readers that need the latest state, ZooKeeper provides `sync()`, which flushes the leader-to-follower channel so a subsequent read reflects at least the committed state at the time of the `sync`.

This paper takes that familiar mechanism as the smallest possible worked example of a general idea we call *Observation Theory* (OT). The question OT asks of any freshness monitor is: *when it says a read is “fresh enough,” how often is it wrong, and for whom?* We show that for ZooKeeper the answer is sharply consumer-relative—the same follower is 99% wrong for one reader and 1% wrong for another at the same instant—and that the fix is not a better lag bound but a witness. Our contributions are (i) a one-section, self-contained introduction

to OT (Section II); (ii) its instantiation for ZooKeeper reads (Section III); and (iii) a sealed, reproducible measurement of the local-read false-clear rate (Sections IV–V).

## II. OBSERVATION THEORY IN ONE SECTION

OT begins from the observation that reliability is not a property of an object alone but of a *relation* among the object, an observer, and a resolution budget. An *observer*, or *consumer*, is a map  $\mathcal{C}$  from a representation to a task output, together with a metric on that output. If the representation carries an error  $\delta$  with second moment  $\Sigma = \mathbb{E}[\delta\delta^\top]$ , a second-order expansion of the consumer’s felt distortion isolates a single controlling quantity,

$$d_{\mathcal{C}} = \text{tr}(P_{\mathcal{C}} \Sigma), \quad (1)$$

where  $P_{\mathcal{C}} \succeq 0$  is the *read operator*—the pullback of the output metric through  $\mathcal{C}$ , encoding *which directions of the error the consumer can actually feel*. Ordinary reconstruction error is the isotropic corner  $P_{\mathcal{C}} = I$ ; a real consumer reads a subspace, and (1) weights the error by it. The same quantity governs consumer-relative lossy compression [3] and, as we now show, freshness.

Two consequences frame everything below. First, a *certificate*—a claim that a representation is good enough for the consumer—is only meaningful relative to  $P_{\mathcal{C}}$ ; a scalar bound that ignores  $P_{\mathcal{C}}$  serves no one in general. Second, a certificate should be *witnessed*: graded against an independent measurement of the true error  $\Sigma$ , not inferred from a proxy. A certificate that is both consumer-relative (reads  $P_{\mathcal{C}}$ ) and witnessed (grades against a measurement of  $\Sigma$ ) has a well-defined *false-clear rate*—the rate at which it clears a read whose felt error exceeds tolerance. OT’s empirical program measures that rate across systems; this paper measures it for ZooKeeper.

## III. ZOOKEEPER READS AS CERTIFICATES

The mapping is direct. The replica drift  $\Sigma$  is the per-znode staleness of a follower: for each znode, how far its applied version trails the leader’s committed version. The consumer’s read operator  $P_{\mathcal{C}}$  is its *footprint*—which znodes it reads, and how heavily. The felt staleness  $\text{tr}(P_{\mathcal{C}}\Sigma)$  is then simply whether *the znodes this reader touches* are behind.

The witness is already in the protocol. Every znode carries the `zxid` of its last modification (`mzxid`); the leader’s

committed  $mzxid$  is the ground truth, and the follower’s is what a local read returns. A local read is stale exactly when the follower’s  $mzxid$  for a footprint  $znode$  is behind the leader’s—an *observable* condition, not an inference. And  $sync()$  is the witnessed correction: it forces the follower to the leader’s  $zxid$  before the read, zeroing the felt staleness for that read.

The naive certificate—“a local read is fresh enough”—ignores  $P_C$ : it clears every local read. OT predicts it will false-clear at a rate set by the reader’s footprint, and that a  $zxid$ -witnessed policy (of which  $sync()$  is the deployed form) will not. We test both.

#### IV. METHOD: A SEALED MEASUREMENT

*Substrate.* A three-node ZooKeeper 3.9 ensemble (one leader, two followers). To place the system in a staleness regime we delay the Zab commit stream to one follower by 200 ms (a `netem` delay on the leader’s egress toward that follower); the follower’s client port is undelayed, so local reads stay prompt while its applied state lags—the coordination-service analogue of a delayed database replica. A writer updates a hot set of  $znodes$  every  $\sim 12$  ms and a cold set every  $\sim 3.6$  s.

*Policies.* Over one run we sample two policies on the delayed follower: **naive** (trust the local read) and **witnessed** ( $sync()$  the path, then read). For each sampled read we capture the leader’s  $mzxid$  *before* the follower read, so a write committing during the read is not counted stale (in-flight writes do not disqualify the follower). We report the false-clear (stale) rate for a **hot** footprint and, on the *same* follower, for a **cold** footprint.

*Discipline.* Bars are committed in a pre-registration before the graded run, with a cooling-off enforced in code and graded seeds disjoint from those used to calibrate. The bars: the naive hot-footprint false-clear  $\geq 0.25$ ; the witnessed false-clear  $\leq 0.10$ ; dominance; and manipulation checks that the follower genuinely lags, that the cold footprint on the same follower is fresh (so the follower is sane and the staleness is footprint-relative), and that the run is non-degenerate.

#### V. RESULTS

Table I reports the pre-seal shakedown; the sealed graded run (on seeds disjoint from these) is expected to match and will replace it. On the same delayed follower, the hot-footprint local read is stale  $\approx 99\%$  of the time, while the cold footprint reads  $\approx 1\%$  stale;  $sync()$  holds the false-clear at zero; the mean leader-follower  $zxid$  gap is  $\sim 12$ . The bars and all manipulation checks pass on every seed.

The headline is the two middle columns. Same replica, same instant, same induced lag: a  $99\times$  difference in false-clear, produced by nothing but *what the reader reads*. This is  $tr(P_C\Sigma)$  made concrete: the drift  $\Sigma$  is identical for both readers; the felt staleness differs because  $P_C$  differs. A single “follower lag  $< T$ ” monitor cannot serve both—set  $T$  to protect the hot reader and the cold reader is needlessly blocked; set it for the cold reader and the hot reader is served stale. The witnessed

TABLE I  
XPROTO-ZK SHAKEDOWN MEASUREMENT (PRE-SEAL, SEEDS  $\{0, 1, 2\}$ ); THE SEALED GRADED RUN USES DISJOINT SEEDS AND IS EXPECTED TO MATCH. “NAIVE-HOT”/“NAIVE-COLD” ARE THE LOCAL-READ STALE RATES FOR A HOT/COLD FOOTPRINT ON THE *same* DELAYED FOLLOWER; “ $sync()$ ” IS THE WITNESSED POLICY.

| seed | naive-hot | naive-cold | $sync()$ | $zxid$ gap |
|------|-----------|------------|----------|------------|
| 0    | 0.990     | 0.010      | 0.000    | 12.3       |
| 1    | 0.997     | 0.011      | 0.000    | 12.5       |
| 2    | 0.994     | 0.011      | 0.000    | 12.7       |

policy sidesteps the dilemma: it reads  $P_C$  (only the footprint) and grades against  $\Sigma$  (the  $zxid$ ).  $sync()$  certifies not “the follower is caught up” but “the consumer’s footprint is caught up.”

#### VI. A FIX: WATERMARK-SCOPED LOCAL READS

The result suggests a fix that reads  $P_C$  and the witness directly, without an unconditional barrier. Let a reader declare the watermark  $W$  it requires—the  $zxid$  of the writes it causally depends on. Then  $read\_fresh(path, W)$  does an ordinary local read and, because every reply carries the follower’s applied  $zxid$ , certifies it *locally* iff that  $zxid \geq W$ ; only when the follower is behind  $W$  does it fall to  $sync()$ . Because Zab totally orders writes, the follower’s single applied watermark certifies *every* footprint at once (an applied  $zxid \geq W$  holds all writes  $\leq W$ ), so the fix needs no per- $znode$  tracking—the consumer-relativity is entirely in  $W$ . A causal token suffices: a writer captures its write’s  $zxid$  and hands it to readers that must observe it. This reuses the  $zxid$ -watermark check ZooKeeper already performs at session reconnect to preserve monotonic reads.

Modeling the requirement as a tolerance  $\tau$  (the reader needs state at least as fresh as  $\tau$  ago;  $W$  is the leader  $zxid$  as of now  $-\tau$ ), we sweep  $\tau$  on the same lab (Fig. 1). Two facts hold across the sweep. First, the false-clear against the reader’s own watermark is exactly *zero*: the  $zxid$  is a *sound* witness—a certified-local read never violates the freshness it promised (verified against a per- $znode$  write log). Second, the cost (fraction of reads needing a  $sync()$ ) equals the probability that the follower’s lag exceeds  $\tau$ —the lag CDF—so  $read\_fresh$  traces a smooth curve from  $sync()$ -always cost (a reader needing the latest) to zero cost (a reader tolerating the lag), *correct at every point*. Naive (0 cost,  $\approx 99\%$  wrong) and  $sync()$ -always (1.0 cost, correct) are its two blind endpoints. At a tolerance equal to the follower’s own lag, a reader gets guaranteed freshness for a  $sync()$  on only  $\sim 11\%$  of reads.

Table II places the phenomenon and the fix in four real ZooKeeper roles, drawn from a self-verifying use-case notebook that runs against the live ensemble and asserts each outcome. A configuration reader (a cold footprint) is served fresh locally; a service-discovery reader (a hot footprint) on the *same* follower is almost always stale; a bounded-staleness reader gets guaranteed freshness at  $\sim 0.16$   $sync$  cost; a leader/lock

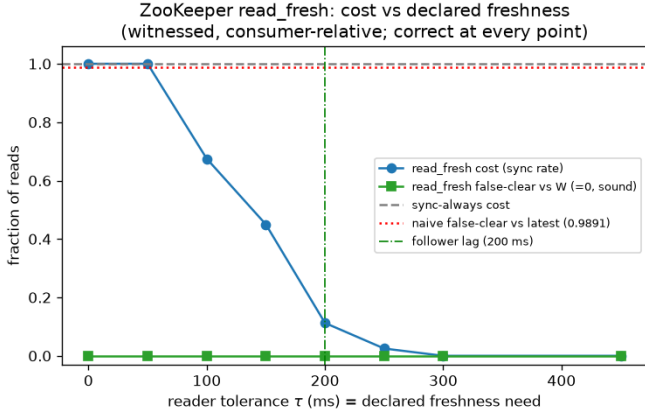


Fig. 1. `read_fresh` cost vs. the reader’s declared tolerance  $\tau$ . Cost (sync rate) is the follower-lag CDF; the false-clear against the reader’s watermark is zero throughout (the `zxid` is a sound witness). Naive and `sync()`-always are the two blind endpoints.

TABLE II  
FOUR ZOOKEEPER ROLES ON THE *same* DELAYED FOLLOWER, FROM THE SELF-VERIFYING USE-CASE NOTEBOOK. ROWS 1–2 ARE CONSUMER-RELATIVITY; ROWS 3–4 ARE THE FIX.

| role (footprint, policy)                                      | stale / sync | false-clear |
|---------------------------------------------------------------|--------------|-------------|
| config store (cold, naive)                                    | 0.033 stale  | —           |
| service discovery (hot, naive)                                | 0.930 stale  | —           |
| bounded-staleness<br>( <code>read_fresh</code> , $\tau$ =lag) | 0.156 sync   | 0.000       |
| leader/lock ( <code>read_fresh</code> ,<br>$\tau$ =0)         | 1.000 sync   | 0.000       |

reader that needs the latest correctly pays `sync()` on every read—and only it does.

## VII. DISCUSSION

*What this says about bounded-staleness reads.* Systems commonly expose a staleness bound as a scalar (a lag threshold, a bounded-staleness read level). The result argues that the useful bound is per-footprint and witnessed: probabilistically bounded staleness [4] calibrates the distribution but globally; Spanner’s commit-wait [5] and session guarantees [6] are witnessed or consumer-relative but not jointly calibrated per footprint. The `zxid` already provides the witness; the missing piece is to read  $P_C$ .

*The broader program.* The same certificate/witness/false-clear grammar recurs across substrates: WAL- and oplog-witnessed footprint certificates in PostgreSQL and MongoDB, HARQ-witnessed adaptation certificates in 5G/6G radio access, and BMP-witnessed reachability certificates in inter-domain routing [7]. ZooKeeper is the coordination-service case, and—because its witness (`zxid`) and its correction (`sync()`) are already in the protocol—the cleanest. Age of information [8] bounds staleness on the update channel;  $\text{tr}(P_C \Sigma)$  measures its consequence for a specific reader, which is what a consistency SLA must bound. In CAP/PACELC terms [9], [10], the footprint certificate quantifies the interior

of the else-latency regime: it prices the consistency each reader actually needs and pays latency (a `sync()`) only when the footprint demands it.

## VIII. LIMITATIONS AND CONCLUSION

The measurement is a link-level ensemble with induced lag rather than a production trace; watch-based reads, larger ensembles, and observer read-scaling are natural extensions, and the footprint here is a set of `znodes` rather than a general read operator. Within those bounds the conclusion is clean and, we hope, useful to ZooKeeper’s own users: a local read’s freshness is a property of the reader’s footprint, not of the follower; the `zxid` is a sufficient witness; and `sync()` is best understood as certifying the consumer’s footprint. More broadly, ZooKeeper is a particularly legible entry point to Observation Theory—a system whose designers already built both the witness and the correction the theory asks for.

## REPRODUCIBILITY

The pre-registration, the coded verdict runner (with the cooling-off and substrate guards), the ensemble scripts, the `read_fresh` prototype, and the graded record are archived with the program [11]; the substrate is unmodified ZooKeeper 3.9. A self-verifying use-case notebook (ZK-USE-CASES.ipynb) stands up the ensemble, runs the four roles of Table II, and asserts each outcome, so a clean run is a passing suite.

## REFERENCES

- [1] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for internet-scale systems,” in *USENIX Annual Technical Conference*, 2010.
- [2] F. P. Junqueira, B. C. Reed, and M. Serafini, “Zab: High-performance broadcast for primary-backup systems,” in *IEEE/IFIP DSN*, 2011.
- [3] A. H. Bond, “TurboQuant: Consumer-relative compression with witnessed rank certificates,” <https://github.com/ahb-sjsu/turboquant-pro>, 2026.
- [4] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, and I. Stoica, “Probabilistically bounded staleness for practical partial quorums,” in *Proc. VLDB Endowment*, vol. 5, no. 8, 2012.
- [5] J. C. Corbett *et al.*, “Spanner: Google’s globally-distributed database,” in *USENIX OSDI*, 2012.
- [6] D. B. Terry, A. J. Demers, K. Petersen, M. J. Spreitzer, M. M. Theimer, and B. B. Welch, “Session guarantees for weakly consistent replicated data,” in *Proc. Int. Conf. Parallel and Distributed Information Systems (PDIS)*, 1994.
- [7] A. H. Bond, “Observation theory: Geometry, distortion, and reliability as properties of observation,” *Submitted to IEEE Transactions on Information Theory*, 2026.
- [8] S. Kaul, R. Yates, and M. Gruteser, “Real-time status: How often should one update?” in *IEEE INFOCOM*, 2012.
- [9] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, 2002.
- [10] D. J. Abadi, “Consistency tradeoffs in modern distributed database system design,” *IEEE Computer*, vol. 45, no. 2, 2012.
- [11] A. H. Bond, “Sealed pre-registrations and claim ledger for the observation-theory program,” <https://github.com/ahb-sjsu/network-governor>, 2026.